# Conservation Map Integration Plan

**Date:** 2026-04-15 **Status:** Draft **Author:** Product Manager Agent **Input Documents:**

- Architecture Proposal: `docs/planning/architecture/conservation-map-proposal.md`
- BA Analysis: `docs/planning/business-analysis/conservation-map-analysis.md`
- Existing gate specs: Gate 02 through Gate 07

**Purpose:** Define the specific additions to existing gates required to deliver the Interactive Conservation Map feature for the ECA Workshop (June 1, 2026). This document does not rewrite existing gate specs -- it specifies additive sections to be appended to each affected gate.

---

## Table of Contents


1. Gate 02 Amendment: Spatial Models
2. Gate 05 Amendment: Map API Endpoints
3. Gate 06 Amendment: Seed Data and Reference Layers
4. Gate 07 Amendment: Conservation Map Page (FE-07-5)
5. Dependency Map
6. Effort Assessment
7. Updated Exit Criteria by Gate
8. Risks and Open Questions


---

## Gate 02 Amendment: Spatial Models

**Insert after:** Model Specifications section (after `integration.SyncJob`) **Effort vs. existing gate scope:** Small (S). Three model definitions with migrations. Gate 02 is already model-heavy; these add ~30% more model code but no new conceptual complexity.

### Scope Change: Remove PostGIS Geometry Exclusion

The existing Gate 02 Out of Scope section states: "PostGIS geometry fields -- no spatial data in MVP models." **Remove this line.** PostGIS is already configured in Gate 01 (`django.contrib.gis` in `INSTALLED_APPS`, PostGIS engine). The three new models below use `PointField` and `MultiPolygonField` from `django.contrib.gis.db.models`.

### Django Applications Table Addition

| App | New Models in This Gate |
|---|---|
| `species` | SpeciesLocality, Watershed, ProtectedArea |

### New Model: `species.SpeciesLocality`

| Field | Type | Notes |
|---|---|---|
| `id` | AutoField | PK |
| `species` | FK -> Species | `on_delete=CASCADE`; indexed |
| `locality_name` | CharField(300) | e.g., "Amboaboa River at Antsirabe confluence" |

| | | |
|---|---|---|
| `location` | PointField(srid=4326) | Exact coordinates (WGS 84) |
| `location_generalized` | PointField(srid=4326), nullable | Pre-computed generalized coordinates for public display. Null if not sensitive. |
| `water_body` | CharField(200), blank | Name of river, lake, etc. |
| `water_body_type` | CharField, enum | `river` / `lake` / `stream` / `cave_system` / `wetland` / `estuary` |
| `drainage_basin` | FK -> Watershed, nullable | Enables efficient "all species in this watershed" queries |
| `drainage_basin_name` | CharField(200), blank | Denormalized basin name. Auto-populated from FK on save. |
| `locality_type` | CharField, enum | `type_locality` / `collection_record` / `literature_record` / `observation` |
| `presence_status` | CharField, enum | `present` / `historically_present_extirpated` / `presence_unknown` / `reintroduced` |
| `source_citation` | TextField | Required. Provenance is non-negotiable for biodiversity data. |
| `year_collected` | IntegerField, nullable | Year of the record, not the publication year. |
| `collector` | CharField(200), blank | Collector or observer name(s). |
| `coordinate_precision` | CharField, enum | `exact` / `approximate` / `locality_centroid` / `water_body_centroid` |
| `is_sensitive` | BooleanField, default False | If True, `location_generalized` is served to Tier 1-2 users instead of `location`. |
| `notes` | TextField, blank | |
| `created_at` | DateTimeField, auto_now_add | |
| `updated_at` | DateTimeField, auto_now | |

**Constraints:**

- `unique_together = (species, location, locality_type)`
- `save()` override: if `is_sensitive` and `location` is not null, compute `location_generalized` as `Point(round(lng, 1), round(lat, 1))`. If not sensitive, set `location_generalized = None`.
- `save()` override: if `drainage_basin` FK is set and `drainage_basin_name` is empty, populate from `drainage_basin.name`.

### New Model: `species.Watershed`

| Field | Type | Notes |
|---|---|---|
| `id` | AutoField | PK |

| | | |
|---|---|---|
| `hybas_id` | BigIntegerField, unique | HydroBASINS feature ID |
| `name` | CharField(200) | Basin name; fallback "Unnamed basin [hybas_id]" for unnamed features |
| `pfafstetter_level` | IntegerField | Pfafstetter coding level (MVP loads level 6 only) |
| `pfafstetter_code` | BigIntegerField | Pfafstetter basin code |
| `parent_basin` | FK -> self, nullable | For hierarchical basin navigation |
| `area_sq_km` | DecimalField(12,2), nullable | Basin area from HydroBASINS attributes |
| `geometry` | MultiPolygonField(srid=4326) | Basin boundary polygon |
| `created_at` | DateTimeField, auto_now_add | |

### New Model: `species.ProtectedArea`

| Field | Type | Notes |
|---|---|---|
| `id` | AutoField | PK |
| `wdpa_id` | IntegerField, unique | WDPA feature ID |
| `name` | CharField(300) | Protected area name |
| `designation` | CharField(200) | e.g., "National Park", "Special Reserve" |
| `iucn_category` | CharField(20), blank | IUCN PA category (Ia, Ib, II, III, IV, V, VI) |
| `status` | CharField(100) | e.g., "Designated", "Proposed" |
| `status_year` | IntegerField, nullable | Year of designation |
| `area_km2` | DecimalField(12,2), nullable | Reported area |
| `geometry` | MultiPolygonField(srid=4326) | PA boundary |
| `created_at` | DateTimeField, auto_now_add | |

### Gate 02 Acceptance Criteria Additions

**Given** a SpeciesLocality record with valid coordinates (-18.91, 47.52) **When** the record is saved **Then** the PointField stores the coordinates correctly; querying with `ST_Contains` against a polygon that includes those coordinates returns the record

**Given** two SpeciesLocality records with the same `species`, `location`, and `locality_type` **When** the second is saved **Then** a database-level unique constraint violation is raised

**Given** a SpeciesLocality record with `is_sensitive = True` and `location = Point(47.5234, -18.9156)` **When** the record is saved **Then** `location_generalized` is auto-computed as `Point(47.5, -18.9)` (rounded to 0.1 degree)

**Given** a SpeciesLocality record with `is_sensitive = False` **When** the record is saved **Then** `location_generalized` is null

**Given** a Watershed record with a valid MultiPolygon geometry **When** the record is saved with `hybas_id = 1060000010` **Then** the record saves without error; a second record with the same `hybas_id` raises a unique constraint violation

**Given** a ProtectedArea record with `wdpa_id = 303847` **When** the record is saved **Then** the record saves without error; a second record with the same `wdpa_id` raises a unique constraint violation

---

## Gate 05 Amendment: Map API Endpoints

**Insert after:** BE-05-6 (Conservation Dashboard) in the User Stories section **Effort vs. existing gate scope:** Medium (M). Three new endpoints with GeoJSON serialization. Adds `djangorestframework-gis` as a dependency. Gate 05 currently has 6 stories; this adds 3, increasing scope by ~50%. However, the map endpoints are simpler than the tier-gated species endpoints because they are all public (Tier 1) with no tier-conditional field visibility.

### New Dependency

Add `djangorestframework-gis` to `requirements.txt`. This is the standard DRF extension for GeoJSON serialization. Add `rest_framework_gis` to `INSTALLED_APPS`.

### Endpoint Table Additions

| Method | Path | Min Tier | Description |
|---|---|---|---|
| `GET` | `/api/v1/map/localities/` | 1 (public) | Species localities as GeoJSON FeatureCollection |
| `GET` | `/api/v1/map/watersheds/` | 1 (public) | Watershed list (no geometry; for filters and info panels) |
| `GET` | `/api/v1/map/summary/` | 1 (public) | Map aggregate statistics |

### BE-05-7: Species Localities GeoJSON Endpoint

**As** the Next.js map page, **I want** `GET /api/v1/map/localities/` to return a GeoJSON FeatureCollection of species localities, **so that** I can render locality markers on the Leaflet map.

**Query parameters:**

- `species_id` (integer) -- filter to a single species
- `family` (string) -- filter by family name
- `iucn_status` (string) -- filter by IUCN category code
- `watershed_id` (integer) -- filter by Watershed FK
- `locality_type` (string) -- filter by locality type enum
- `presence_status` (string) -- filter by presence status enum
- `coordinate_precision` (string) -- filter by coordinate precision enum
- `bbox` (string) -- bounding box: `min_lng,min_lat,max_lng,max_lat`

**Response (200 OK):**

```json
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [47.52, -18.91]
      },
      "properties": {
        "id": 42,
        "species_id": 7,
        "scientific_name": "Pachypanchax sakaramyi",
        "family": "Aplocheilidae",
        "iucn_status": "EN",
        "locality_name": "Sakaramy River near Joffreville",
        "locality_type": "type_locality",
        "presence_status": "present",
        "water_body": "Sakaramy River",
        "water_body_type": "river",
        "drainage_basin_name": "Sambirano",
        "year_collected": 1928,
        "coordinate_precision": "exact",
        "source_citation": "Holly, 1928"
      }
    }
  ]
}
```

**Tier behavior at MVP:** All requests are Tier 1 (unauthenticated). The serializer serves `location_generalized` for records where `is_sensitive = True`, and `location` for non-sensitive records. The `geometry` field in the GeoJSON response is always the appropriate coordinate for the requester's tier.

**Serializer:** `SpeciesLocalityGeoSerializer` using `rest_framework_gis.serializers.GeoFeatureModelSerializer`. Custom `get_geometry` method implements the tier-aware coordinate selection.

**Performance:**

- Use `select_related('species', 'drainage_basin')` to avoid N+1 queries
- Use `only()` to limit fields to those needed for serialization
- Cache full unfiltered GeoJSON response in Redis (5-minute TTL); invalidate on SpeciesLocality save/delete signals
- Filtered responses are not cached at MVP

**Acceptance Criteria:**

**Given** a GET to `/api/v1/map/localities/` with no parameters **When** the database contains 50 locality records **Then** a GeoJSON FeatureCollection with 50 features is returned; each feature has `type: "Point"` geometry and all specified properties

**Given** a GET to `/api/v1/map/localities/?species_id=7` **When** species 7 has 3 locality records **Then** the response contains exactly 3 features, all with `properties.species_id = 7`

**Given** a GET to `/api/v1/map/localities/?iucn_status=CR&family=Bedotiidae` **When** multiple filters are applied simultaneously **Then** only localities for CR-status Bedotiidae species are returned (filters are AND-combined)

**Given** a GET to `/api/v1/map/localities/` and a SpeciesLocality record has `is_sensitive = True` **When** the response is returned **Then** the geometry coordinates for that feature are the generalized coordinates (rounded to 0.1 degree), not the exact coordinates

**Given** a GET to `/api/v1/map/localities/?bbox=46.0,-20.0,48.0,-18.0` **When** the bbox filter is applied **Then** only localities with coordinates within the bounding box are returned

**Given** a GET to `/api/v1/map/localities/` and the database has zero SpeciesLocality records **When** the response is returned **Then** HTTP 200 with `{"type": "FeatureCollection", "features": []}` -- not a 404

---

### BE-05-8: Watershed List Endpoint

**As** the Next.js map page, **I want** `GET /api/v1/map/watersheds/` to return a list of watersheds with species counts, **so that** I can populate the watershed filter dropdown and watershed info panels.

**Response (200 OK):**

```
[
  {
    "id": 1,
    "name": "Betsiboka",
    "pfafstetter_level": 6,
    "area_sq_km": 48900.00,
    "species_count": 12
  }
]
```

`species_count` is annotated via a subquery counting distinct species in SpeciesLocality records for that watershed.

**Note:** Watershed geometries are NOT served through this endpoint. They are served as static GeoJSON files at `/static/map-layers/watersheds.geojson`. This endpoint provides metadata only.

**Acceptance Criteria:**

**Given** a GET to `/api/v1/map/watersheds/` **When** 60 Watershed records exist, 34 of which have at least one SpeciesLocality **Then** all 60 watersheds are returned; watersheds with localities have `species_count > 0`; watersheds without localities have `species_count = 0`

**Given** a GET to `/api/v1/map/watersheds/` and no Watershed records exist **When** the response is returned **Then** HTTP 200 with an empty array `[]`

---

### BE-05-9: Map Summary Endpoint

**As** the Next.js map page, **I want** `GET /api/v1/map/summary/` to return aggregate map statistics, **so that** I can render the statistics bar and communicate data coverage.

**Response (200 OK):**

```
{
  "total_localities": 547,
  "species_with_localities": 72,
  "species_without_localities": 7,
  "watersheds_represented": 34,
  "locality_type_counts": {
    "type_locality": 79,
    "collection_record": 312,
    "literature_record": 98,
    "observation": 58
  },
  "presence_status_counts": {
    "present": 420,
    "historically_present_extirpated": 67,
    "presence_unknown": 45,
    "reintroduced": 15
  }
}
```

Cached in Redis (5-minute TTL), invalidated on SpeciesLocality changes.

**Acceptance Criteria:**

**Given** a GET to `/api/v1/map/summary/` **When** the database contains 547 SpeciesLocality records across 72 species **Then** `total_localities = 547`, `species_with_localities = 72`, `species_without_localities` equals total species minus 72

**Given** a GET to `/api/v1/map/summary/` and the database has zero SpeciesLocality records **When** the response is returned **Then** `total_localities = 0`, `species_with_localities = 0`, all type/status counts are 0 or omitted; HTTP 200 (not an error)

---

### Gate 05 Technical Tasks Additions

- Create `species/serializers_map.py` with `SpeciesLocalityGeoSerializer` (using `GeoFeatureModelSerializer`), `WatershedListSerializer`, `MapSummarySerializer`
- Create `species/views_map.py` with `SpeciesLocalityGeoViewSet`, `WatershedListView`, `MapSummaryView`
- Register URL routes under `/api/v1/map/` namespace in `config/urls.py`
- Add `djangorestframework-gis` to `requirements.txt` and `INSTALLED_APPS`
- Write integration tests: GeoJSON response shape validation, filter combinations, sensitive coordinate redaction, empty state responses
- Add N+1 query tests for the localities endpoint with `select_related`

---

## Gate 06 Amendment: Seed Data and Reference Layers

**Insert after:** Task 4 (Seed Data Management Command) in the Technical Tasks section **Effort vs. existing gate scope:** Medium-Large (M-L). Four new management commands and CSV schema definition. Gate 06 currently has 2 management commands (IUCN client + seed_species); this adds 3 management commands plus CSV validation logic. The reference layer loading involves shapefile processing with GDAL/OGR, which is more complex than CSV parsing.

### Task 5: `load_reference_layers` Management Command

Location: `species/management/commands/load_reference_layers.py`

```
python manage.py load_reference_layers \
    --watersheds data/reference/hydrobasins_madagascar_lev06.shp \
    --protected-areas data/reference/wdpa_madagascar.shp \
    [--simplify 0.001]
```

**Behavior:**

- Loads HydroBASINS shapefile into `Watershed` table, filtering to Madagascar features only
- Loads WDPA shapefile into `ProtectedArea` table, filtering to `ISO3 = 'MDG'`
- Applies `ST_Simplify` at specified tolerance (default 0.001 degrees, ~100m) to reduce vertex counts
- Idempotent: keyed on `hybas_id` (Watershed) and `wdpa_id` (ProtectedArea). Existing records are updated; new records are created; no records are deleted.
- Implements top-down watershed naming: match ~15-20 major basin names from published hydrological maps (Betsiboka, Tsiribihina, Mangoky, Onilahy, Mangoro, Sofia, etc.); auto-generate "Sub-basin of [parent name]" for unnamed features using Pfafstetter hierarchy. Must avoid "Sub-basin of Sub-basin of..." chains by resolving to the nearest named ancestor.
- `--dry-run` flag: reports what would be loaded without writing to database
- Logs: N watersheds created/updated, N protected areas created/updated, total vertex count before/after simplification

**Data source files (must be prepared before running):**

- `data/reference/hydrobasins_madagascar_lev06.shp` -- extracted from HydroSHEDS Africa level 6, filtered to Madagascar bounding box
- `data/reference/wdpa_madagascar.shp` -- extracted from WDPA monthly download, filtered to `ISO3 = 'MDG'`

### Task 6: `seed_localities` Management Command

Location: `species/management/commands/seed_localities.py`

```
python manage.py seed_localities \
    --csv data/seed/madagascar_freshwater_fish_localities_seed.csv \
    [--dry-run]
```

**CSV Schema:**

| Column | Maps to Field | Required | Validation |
|---|---|---|---|
| `scientific_name` | FK lookup -> Species | Yes | Must match `Species.scientific_name` exactl not found |
| `locality_name` | `locality_name` | Yes | Non-empty string |

| | | | |
|---|---|---|---|
| `latitude` | `location` (y) | Yes | Between -26.0 and -11.5 (Madagascar exte |
| `longitude` | `location` (x) | Yes | Between 43.0 and 51.0 (Madagascar exten |
| `water_body` | `water_body` | No | Free text |
| `water_body_type` | `water_body_type` | No | Enum: `river/lake/stream/cave_system/wetlan` |
| `locality_type` | `locality_type` | Yes | Enum: `type_locality/collection_record/lit` |
| `presence_status` | `presence_status` | Yes | Enum: `present/historically_present_extirpated/pr` |
| `coordinate_precision` | `coordinate_precision` | Yes | Enum: `exact/approximate/locality_centroid/` |
| `source_citation` | `source_citation` | Yes | Non-empty string |
| `year_collected` | `year_collected` | No | Integer year; null if blank |
| `collector` | `collector` | No | Free text |
| `is_sensitive` | `is_sensitive` | No | `true/false`; default `false` |
| `notes` | `notes` | No | Free text |

**Columns NOT in CSV (auto-computed on import):**

- `drainage_basin` -- assigned via `ST_Contains` spatial query against loaded Watershed polygons
- `drainage_basin_name` -- populated from FK on save
- `location_generalized` -- computed from `location` + `is_sensitive` on save

**Behavior:**

- Idempotent: keyed on `(species__scientific_name, location, locality_type)`. Existing records are updated; new records are created; records not in the CSV are not deleted.
- `--dry-run` flag: validates CSV and reports what would be created/updated/skipped without writing to database. Reports validation errors per row.
- Logs: N created, N updated, N skipped (validation errors), with error details per row
- Validation warning (not rejection): if `locality_type = "type_locality"` and the matched species has `taxonomic_status = "undescribed_morphospecies"`, log a warning. Undescribed taxa should not have type localities.
- Auto-assign `drainage_basin` FK: for each locality, query `Watershed.objects.filter(geometry__contains=point)`. If exactly one match, assign it. If zero matches (point falls outside all loaded watersheds), set to null and log a warning. If multiple matches (overlapping polygons), assign the smallest by `area_sq_km` and log a notice.

**Prerequisite:** `load_reference_layers` must have been run before `seed_localities` for drainage basin assignment to work. If no Watershed records exist, `seed_localities` logs a warning and sets all `drainage_basin` FKs to null.

### Task 7: `generate_map_layers` Management Command

Location: `species/management/commands/generate_map_layers.py`

```
python manage.py generate_map_layers --output-dir staticfiles/map-layers/
```

**Behavior:**

- Serializes all Watershed records to `staticfiles/map-layers/watersheds.geojson`
- Serializes all ProtectedArea records to `staticfiles/map-layers/protected-areas.geojson`
- GeoJSON includes all attribute fields needed for frontend display (name, designation, area, IUCN category, species count per watershed)
- Applies additional geometry simplification if needed (configurable `--simplify` flag)
- Overwrites existing files (this is a regeneration command, not an append)
- Logs: file path, record count, file size for each generated layer

**Output files:**

- `staticfiles/map-layers/watersheds.geojson` -- served at `/static/map-layers/watersheds.geojson`
- `staticfiles/map-layers/protected-areas.geojson` -- served at `/static/map-layers/protected-areas.geojson`

### Full Seed Execution Order

The following sequence must be followed for a complete data load:

```
# 1. Load reference layers (watersheds + protected areas)
python manage.py load_reference_layers \
    --watersheds data/reference/hydrobasins_madagascar_lev06.shp \
    --protected-areas data/reference/wdpa_madagascar.shp

# 2. Load species (existing Gate 06 command)
python manage.py seed_species --csv data/seed/madagascar_freshwater_fish_seed.csv

# 3. Load species localities (requires both reference layers and species)
python manage.py seed_localities \
    --csv data/seed/madagascar_freshwater_fish_localities_seed.csv

# 4. Generate static GeoJSON files for frontend (requires reference layers loaded)
python manage.py generate_map_layers --output-dir staticfiles/map-layers/
```

**Why this order:**

- Step 1 before Step 3: localities need watershed polygons for `drainage_basin` FK assignment
- Step 2 before Step 3: localities reference species by `scientific_name`
- Step 4 after Step 1: GeoJSON is serialized from the database tables loaded in Step 1

### Data Source Companion Document

The localities CSV must be accompanied by `docs/data-sources/locality-data-sourcing.md`, written by the project lead / data curator. This document describes provenance, geocoding methodology, precision levels, sensitivity decisions, and known gaps for the locality dataset. The architecture proposal (Section 8) provides the recommended structure for this document.

### Gate 06 Acceptance Criteria Additions

**Given** the `load_reference_layers` command is run with valid HydroSHEDS and WDPA shapefiles **When** the command completes **Then** ~60-100 Watershed records and ~120-160 ProtectedArea records exist in the database; all geometries are simplified; major basin names (Betsiboka, Tsiribihina, Mangoky, Onilahy, etc.) are assigned

**Given** the `load_reference_layers` command is run a second time with the same shapefiles **When** the command completes **Then** no duplicate records are created (idempotent on `hybas_id` / `wdpa_id`); existing records are updated

**Given** the `seed_localities` command is run with `--dry-run` **When** the CSV contains 300 rows, 5 of which have invalid coordinates (outside Madagascar bbox) **Then** the command reports 295 valid rows and 5 skipped rows with per-row error details; no database writes occur

**Given** the `seed_localities` command is run with a valid CSV **When** watershed polygons have been loaded via `load_reference_layers` **Then** each locality is assigned a `drainage_basin` FK based on spatial containment; localities outside all watershed polygons have `drainage_basin = null` with a logged warning

**Given** the `seed_localities` CSV contains a row with `locality_type = "type_locality"` for a species with `taxonomic_status = "undescribed_morphospecies"` **When** the command processes that row **Then** the record is imported (not rejected) but a warning is logged: "type_locality for undescribed morphospecies [species_name]"

**Given** the `seed_localities` CSV contains a row with `is_sensitive = true` **When** the record is saved **Then** `location_generalized` is auto-computed as the 0.1-degree snapped coordinate

**Given** the `generate_map_layers` command is run after reference layers are loaded **When** the command completes **Then** `staticfiles/map-layers/watersheds.geojson` and `staticfiles/map-layers/protected-areas.geojson` exist; each file is valid GeoJSON; file sizes are logged

**Given** `seed_localities` is run before `load_reference_layers` **When** no Watershed records exist in the database **Then** all localities are imported with `drainage_basin = null`; a warning is logged: "No watershed records found; drainage_basin FK will not be assigned"

---

## Gate 07 Amendment: Conservation Map Page (FE-07-5)

**Insert after:** FE-07-4 (Navigation and Site Shell) in the User Stories section **Effort vs. existing gate scope:** See Effort Assessment below.

### Navigation Change

**Amend FE-07-4 header links from:** "Species Directory" ( `/species/` ), "Conservation Dashboard" ( `/dashboard/` ), "About"

**To:** "Species Directory" ( `/species/` ), "Map" ( `/map/` ), "Conservation Dashboard" ( `/dashboard/` ), "About"

This follows the logical flow: browse species (directory) -> see where they are (map) -> understand the crisis (dashboard) -> learn about the platform (about).

### API Dependencies Table Addition

| Page | DRF Endpoints Consumed |
|---|---|

| | |
|---|---|
| Conservation Map | `GET /api/v1/map/localities/`, `GET /api/v1/map/watersheds/`, `GET /api/v1/map/summary/` |
| Conservation Map (static) | `GET /static/map-layers/watersheds.geojson`, `GET /static/map-layers/protected-areas.geojson` |

### FE-07-5: Conservation Map Page

**As** a workshop attendee, conservation professional, or public visitor, **I want** to explore an interactive map of Madagascar showing where freshwater fish species have been recorded, **so that** I can understand the geographic distribution of this species group and how it relates to watersheds and protected areas.

**Page:** `/map/` **Rendering:** CSR (client-side rendering). Map interactions are inherently client-side; SSR provides no benefit.

**Frontend dependencies:**

- `leaflet` + `react-leaflet` (React wrapper for Leaflet)
- `leaflet.markercluster` (marker clustering plugin)
- No additional mapping libraries

This story is structured in two tiers. Tier A is must-ship; Tier B ships if time allows.

---

**Tier A: Must-Ship (estimated ~1 week frontend effort)**

**UI elements:**

1. **Full-viewport Leaflet map** centered on Madagascar (approximately -18.9, 47.5, zoom 6)

2. **Base layer switcher** -- OpenStreetMap (default) and ESRI World Imagery (satellite) as switchable base layers via Leaflet's built-in layer control

3. **Species locality point markers:**

   - Fetched from `GET /api/v1/map/localities/`
   - Color-coded by IUCN status using standard IUCN colors: CR=red, EN=orange, VU=yellow, NT=near-green, LC=green, DD=gray, NE=light gray
   - Shape/style varies by locality type: type_locality uses a star or diamond shape; other types use standard circle markers
   - Fill style varies by presence status: solid fill = present; hollow (outline-only) = extirpated; dashed outline = unknown; distinct marker = reintroduced
   - Marker clustering via `leaflet.markercluster` for decluttering at low zoom levels

4. **Marker popups** -- click a locality marker to see:

   - Scientific name (italic, linked to `/species/{id}/`)
   - IUCN status badge
   - Locality name
   - Locality type (e.g., "Type Locality")
   - Presence status
   - Water body name
   - Year collected (if available)
   - Source citation
   - Coordinate precision indicator (e.g., "Location precision: exact")

- For sensitive records with generalized coordinates: "Location generalized to protect sensitive species"

5. **Legend:**
   - Color key for IUCN status (CR, EN, VU, NT, LC, DD, NE with corresponding colors)
   - Shape key for locality type (type_locality visually distinct from others)
   - Fill/outline key for presence status (solid=present, hollow=extirpated, dashed=unknown, distinct for reintroduced)
6. **"View on Map" cross-link from species profiles** -- on `/species/[id]/` pages, add a "View on Map" button that links to `/map/?species_id={id}`. Button is only shown when the species has at least one locality record (requires either a `locality_count` field on the species detail API response, or a lightweight check).

**Tier A Acceptance Criteria:**

**Given** a user visiting `/map/` **When** the page loads **Then** a Leaflet map renders centered on Madagascar with OpenStreetMap tiles; species locality point markers are visible with IUCN color coding; the legend is visible

**Given** a user visiting `/map/` and the localities API returns zero records **When** the page loads **Then** the map renders with the base map and no markers; a message reads "Locality data is being compiled -- check back soon"; no error state is displayed

**Given** a user visiting `/map/` and the localities API is unreachable **When** the page loads **Then** the base map still renders (tiles are from CDN); an inline error message indicates "Species locality data temporarily unavailable"

**Given** a user clicks on a species locality marker **When** the popup opens **Then** all specified popup fields are displayed; the scientific name is a clickable link to the species profile; coordinate precision is shown in the popup text

**Given** multiple markers overlap at the current zoom level **When** the user clicks the cluster **Then** the map zooms in to reveal individual markers (standard markercluster behavior)

**Given** a user clicks the species name link in a marker popup **When** the navigation occurs **Then** the user arrives at `/species/{id}/` for that species

**Given** a user switches from OpenStreetMap to ESRI satellite base layer **When** the switch occurs **Then** the base tiles change; all point markers remain visible and correctly positioned

**Given** a user viewing a species profile at `/species/{id}/` for a species with locality records **When** the page renders **Then** a "View on Map" button is visible

**Given** a user clicks "View on Map" on the profile for *Pachypanchax sakaramyi* **When** the map page loads **Then** the URL is `/map/?species_id={id}`; the map shows only that species' localities; the map auto-zooms to fit the extent of that species' points

**Given** a user viewing a species profile for a species with ZERO locality records **When** the page renders **Then** the "View on Map" button is absent or disabled with tooltip "No locality data available"

**Given** a user arrives at `/map/?species_id=9999` (nonexistent species) **When** the map page loads **Then** the map renders with all localities (no filter); an inline notice reads "Species not found -- showing all localities"

**Given** a user on a tablet (768-1024px viewport) **When** the map page loads **Then** the map fills the available viewport; touch gestures (pinch zoom, pan) work correctly; marker popups fit within the viewport

**Given** a user on a mobile phone (< 768px viewport) **When** the map page loads **Then** the map renders at full width and at least 60% viewport height; the legend is collapsible or minimized

---

**Tier B: Ship If Time Allows (estimated ~1-2 weeks additional frontend effort)**

Tier B features build on Tier A. None of Tier B should be started until Tier A is complete and tested.

**UI elements:**

7. **Watershed polygon overlay** (toggleable, off by default):
    - Loaded lazily from `/static/map-layers/watersheds.geojson` on first toggle
    - Semi-transparent styling; does not obscure point markers beneath
    - Loading indicator while GeoJSON is fetching
    - Click a watershed polygon -> popup/sidebar shows: watershed name, area in km2, list of species in that basin (from already-loaded locality data, filtered client-side by `drainage_basin_name`), each species name linked to profile with IUCN badge
8. **Protected area polygon overlay** (toggleable, off by default):
    - Loaded lazily from `/static/map-layers/protected-areas.geojson` on first toggle
    - Visually distinct from watershed overlay (different color, opacity, border style)
    - Loading indicator while GeoJSON is fetching
    - Point markers remain clickable through polygon overlays
    - Locality markers render above both polygon layers in z-order
9. **Filter panel** (collapsible sidebar):
    - Family dropdown (populated from locality data or species list)
    - IUCN status multi-select
    - Locality type multi-select
    - Presence status multi-select
    - Watershed dropdown (populated from `/api/v1/map/watersheds/`)
    - Coordinate precision multi-select
    - Filters update markers client-side (re-request `/api/v1/map/localities/` with filter params)
    - Filter state reflected in URL query parameters (shareable links)
    - On mobile/tablet: filter panel accessible via button/drawer, not permanently visible
10. **Map statistics bar** (top or bottom of map):
    - "X localities for Y species" (updates with active filters)
    - Data from `/api/v1/map/summary/` for initial load; updated client-side when filters are applied

**Tier B Acceptance Criteria:**

**Given** a user toggles the "Watersheds" layer on for the first time **When** the static GeoJSON is loading **Then** a loading indicator appears on the layer toggle; the map remains interactive; once loaded the polygons render

**Given** a user clicks on a watershed polygon (e.g., Betsiboka) **When** the popup renders **Then** it shows the watershed name, area, and a list of species with localities in that basin; each species name links to its

profile; if no species exist: "No species locality records in this basin"

**Given** a user clicks on an unnamed watershed **When** the popup renders **Then** the name displays as "Sub-basin of [parent name]" or "Basin [Pfafstetter code]" -- never blank, never "null"

**Given** both "Watersheds" and "Protected Areas" layers are toggled on **When** both layers render **Then** they use distinct visual styling (different colors, opacity, borders); point markers render above both polygon layers

**Given** a user on a slow connection (simulated 3G) toggles the protected areas layer **When** loading is in progress **Then** a loading state is visible; the map does not freeze; the user can cancel by toggling the layer off

**Given** a user selects "CR" in the IUCN status filter **When** the filter is applied **Then** only CR-status localities are visible; the statistics bar updates; the URL reflects the filter state

**Given** a user filters by `presence_status = "historically_present_extirpated"` **When** the filter is applied **Then** only extirpated localities are shown; the statistics bar updates

**Given** a user on a tablet with the filter panel open **When** they select a family filter **Then** the filter applies immediately; the map updates; the filter panel remains open for additional selections

**Given** a user on mobile (< 768px) **When** the map page loads **Then** the filter panel is hidden by default; accessible via a button/drawer; statistics bar collapses to essential info

---

### Scope Assessment

| Story / Component | Frontend | Backend | Full-Stack | Complexity |
|---|---|---|---|---|
| Gate 02: Spatial models | | X | | S |
| BE-05-7: Localities GeoJSON endpoint | | X | | M |
| BE-05-8: Watershed list endpoint | | X | | S |
| BE-05-9: Map summary endpoint | | X | | S |
| Gate 06: `load_reference_layers` cmd | | X | | M |
| Gate 06: `seed_localities` cmd | | X | | M |
| Gate 06: `generate_map_layers` cmd | | X | | S |
| FE-07-5 Tier A: Map page (points, popups, legend, clustering, base layers) | X | | | L |
| FE-07-5 Tier A: "View on Map" cross-link from species profile | X | | | S |
| FE-07-5 Tier B: Watershed + PA overlays | X | | | M |
| FE-07-5 Tier B: Filter panel | X | | | M |
| FE-07-5 Tier B: Statistics bar | X | | | S |
| FE-07-4 amendment: Add "Map" to nav | X | | | S |

## Dependency Map

```
Gate 01 (PostGIS configured)
  |
  v
Gate 02: SpeciesLocality, Watershed, ProtectedArea models + migrations
  |
  +---> Gate 03 (no map changes; auth primitives used by Gate 05)
  |       |
  |       v
  |     Gate 04: Admin registrations for 3 new models [NOTE 1]
  |       |
  |       v
  |     Gate 05: BE-05-7 (localities GeoJSON)  --|
  |              BE-05-8 (watershed list)       -+--> Gate 07: FE-07-5
  |              BE-05-9 (map summary)          --|
  |
  +---> Gate 06: load_reference_layers (needs Watershed + PA tables from Gate 02)
  |       |
  |       +---> Gate 06: seed_species (existing; no change)
  |       |       |
  |       |       v
  |       +---> Gate 06: seed_localities (needs species + watersheds loaded)
  |       |
  |       +---> Gate 06: generate_map_layers (needs reference layers loaded)
  |               |
  |               v
  |             Static GeoJSON files available --> Gate 07: FE-07-5 Tier B
  |
  v
Gate 07: FE-07-5 Tier A (needs BE-05-7 endpoint + seeded localities)
  |
  v
Gate 07: FE-07-5 Tier B (needs Tier A + static GeoJSON files + BE-05-8)
```

**NOTE 1:** The architecture proposal specifies admin registrations for SpeciesLocality, Watershed, and ProtectedArea in Gate 04. These registrations are straightforward (see architecture proposal Section 6.2) and follow the existing Gate 04 pattern. Key detail: SpeciesLocality uses `OSMGeoAdmin` for the `location` field (map-based point picker). `location_generalized`, `created_at`, and `updated_at` are read-only fields in admin.

### Dependency Rules

1. **No circular dependencies.** Every dependency arrow points forward (lower gate number to higher, or within a gate from earlier task to later task).
2. **Existing gate progression is not blocked.** The map additions to Gates 02, 05, and 06 can be implemented alongside existing gate work. No existing story is blocked by a map story.
3. **The critical path for the map is:** Gate 02 models -> Gate 06 `load_reference_layers` -> Gate 06 `seed_localities` -> Gate 05 map endpoints -> Gate 07 map page. The localities CSV (data

curation by project lead) runs in parallel with code work and must be ready by approximately May 25.

4. **Tier B depends on Tier A.** No Tier B work should begin until Tier A is code-complete and manually tested.
5. **The "View on Map" cross-link (part of Tier A)** depends on the `/map/` page existing. Implement after the map page itself.

---

## Effort Assessment

| Gate | Existing Scope | Map Addition Scope | Map % of Gate | Risk Flag |
|---|---|---|---|---|
| Gate 02 | 8 models + migrations | +3 models with spatial fields | ~25% increase | None. Spatial fields add some complexity but the models are simple. |
| Gate 04 | ~6 admin registrations | +3 admin registrations | ~30% increase | None. Boilerplate admin config. |
| Gate 05 | 6 endpoints + serializers | +3 endpoints + GeoJSON serializer | ~40% increase | **Moderate.** `djangorestframework-gis` is a new dependency; GeoJSON serialization has a different pattern than standard DRF. Budget extra time for GeoSerializer learning curve. |
| Gate 06 | 2 management commands (IUCN client + seed_species) | +3 management commands (load_reference_layers, seed_localities, generate_map_layers) | ~60% increase | **Significant.** `load_reference_layers` involves shapefile parsing with GDAL, spatial queries, and the naming strategy. `seed_localities` involves spatial containment queries. These are more complex than CSV parsing. The localities CSV itself is a parallel data curation dependency on the critical path. |
| Gate 07 | 3 pages + nav + SSR + deploy | +1 page (map) + nav amendment + cross-link | ~30% of frontend effort (Tier A only); ~50% if Tier B ships | **Moderate-High.** Leaflet integration is the most complex single frontend task. However, Tier A is deliberately scoped to be achievable in ~1 week. The Tier B fallback absorbs schedule pressure. |

### June 1, 2026 Deadline Assessment

The architecture proposal estimates a 7-week implementation sequence. With the map feature integrated:

- **Weeks 1-2:** Gates 02-04 with spatial model additions. Low risk.
- **Week 3:** Gate 05 with map API endpoints. Moderate risk (new dependency).
- **Week 4:** Gate 06 with reference layer loading + seed commands. Moderate risk (shapefile processing).
- **Weeks 5-6:** Gate 07 frontend including map page Tier A. High risk (Leaflet integration + all other pages).
- **Week 7:** Polish, testing, Tier B if time allows, deploy.

**Verdict:** The schedule is tight but feasible IF:

1. The localities CSV is ready by May 25 (one week before workshop)
2. Tier B is treated as genuinely optional (not planned into the schedule)
3. Gate 06's reference layer commands are developed early (they do not depend on Gate 05)

**Flag:** Gate 06 scope increase is the most concerning. The three new management commands with spatial processing add significant complexity to a gate that also delivers Celery configuration and IUCN sync. Consider whether `load_reference_layers` and `generate_map_layers` can be started as soon as Gate 02 models are migrated, before Gate 03/04/05 are complete (they have no dependency on auth or admin).

---

## Updated Exit Criteria by Gate

### Gate 02 Additional Exit Criteria

- SpeciesLocality, Watershed, and ProtectedArea migrations apply cleanly on a fresh PostGIS database
- Spatial field tests pass: PointField and MultiPolygonField accept valid geometries; unique constraints on `hybas_id`, `wdpa_id`, and `(species, location, locality_type)` are enforced
- `save()` override on SpeciesLocality correctly computes `location_generalized` for sensitive records and populates `drainage_basin_name` from FK

### Gate 05 Additional Exit Criteria

- `/api/v1/map/localities/` returns valid GeoJSON FeatureCollection with correct response shape
- Sensitive record coordinates are redacted (generalized coordinates served for `is_sensitive = True` records)
- All filter parameters work correctly (species_id, family, iucn_status, watershed_id, locality_type, presence_status, coordinate_precision, bbox)
- Empty state returns 200 with empty FeatureCollection, not 404
- N+1 query tests pass for localities endpoint with `select_related`
- `djangorestframework-gis` is installed and configured

### Gate 06 Additional Exit Criteria

- `load_reference_layers` loads HydroSHEDS and WDPA data correctly; ~60-100 watersheds and ~120-160 protected areas exist; major basins are named
- `seed_localities` loads the CSV correctly; drainage basin FK assignment works via spatial containment; `--dry-run` validates without writing
- `generate_map_layers` produces valid GeoJSON files in `staticfiles/map-layers/`
- Full seed execution order runs without error: `load_reference_layers` -> `seed_species` -> `seed_localities` -> `generate_map_layers`

- Invoke **@test-writer** to write tests for: coordinate validation (outside Madagascar bbox), idempotent re-import, spatial FK assignment, sensitive coordinate generalization, type_locality + undescribed_morphospecies warning

### Gate 07 Additional Exit Criteria

- `/map/` page renders with seeded locality data; markers are color-coded by IUCN status
- Legend is visible and accurate
- Marker popups display all specified fields including coordinate precision
- "View on Map" link appears on species profiles with localities; absent for species without localities
- Map page is responsive on tablet (768-1024px) and mobile (< 768px) viewports
- Navigation header includes "Map" link in the correct position
- Empty state (zero localities) displays graceful message, not an error
- API failure state shows base map with inline error message
- **If Tier B ships:** filter panel works; watershed/PA overlays load lazily; statistics bar updates with filters
- Invoke **@test-writer** to write adversarial frontend tests: empty locality response, malformed GeoJSON, species_id filter with nonexistent ID, slow-loading reference layers, cluster click behavior
- Invoke **@ux-reviewer** for map page usability: is the legend comprehensible? Are marker colors distinguishable? Is the popup information hierarchy clear? Does the mobile layout work for workshop tablet demos?

---

## Risks and Open Questions

### Risks

1. **Localities CSV is on the critical path and does not yet exist.** Mitigation: develop against synthetic test data; treat the real CSV as a late Gate 06 deliverable. The "thin seed" strategy (type localities only, ~79 records) provides a meaningful fallback if full curation is not complete by May 25.

2. **Gate 06 scope increase is substantial.** Three new management commands with spatial processing on top of Celery + IUCN sync. Mitigation: `load_reference_layers` and `generate_map_layers` can be started immediately after Gate 02 models are migrated (no dependency on Gates 03-05). Parallelize where possible.

3. **Leaflet integration is the highest-complexity frontend task.** The map page has more interactive state management than any other Gate 07 page. Mitigation: Tier A/B split ensures the minimum viable map is achievable in ~1 week. Tier B is genuinely optional.

4. **Reference layer GeoJSON file sizes.** Protected area polygons may be 5-15 MB even after simplification. Mitigation: lazy loading (layers off by default); geometry simplification at load time; consider TopoJSON conversion if testing reveals performance issues.

5. **GDAL/OGR dependency for shapefile processing.** The `load_reference_layers` command requires GDAL, which is already a PostGIS dependency but may need explicit Python bindings (`GDAL` package or `fiona`). Verify this is available in the Docker image.

### Open Questions

None. All questions raised in the BA analysis have been resolved (see BA analysis Section 7). All architecture decisions have been made (see architecture proposal Section 9).